National University of Computer and Emerging Sciences

Web Programming

PROJECT

Shahmeer Atif 23I-0711
Muhammad Umar 23I-0782

Web Programming Final Project Report: Inkblot

Project Title: Inkblot – Real-Time Multiplayer Drawing & Guessing Game

Institution: FAST NUCES, Islamabad

---

1. Executive Summary

Inkblot is a highly interactive, real-time web application inspired by Skribbl.io. It allows users to create private or public rooms, invite friends, draw prompts on a synchronized canvas, and compete in real-time by guessing the drawn words. The project was built to satisfy all requirements of the Web Programming Final Project rubric, demonstrating proficiency in modern full-stack web development, real-time WebSocket communication, role-based access control (RBAC), and responsive UI/UX design.

---

2. Technology Stack & Architecture

To achieve the "zero-lag" feel required for a multiplayer game while maintaining a free-tier deployment, the project utilizes a decoupled architecture:

- **Frontend UI & API Routes:** Built with **Next.js (App Router)** and React. Deployed on **Vercel** for fast, serverless delivery.
- **Real-Time Game Engine:** A standalone **Node.js + Socket.IO** server deployed on **Railway**. This prevents the WebSocket connection drops inherent to Vercel's serverless architecture.
- **Database: MongoDB Atlas** (Mongoose) for persistent storage of users, friends, game invites, and room settings.
- **Authentication:** Custom JWT-based authentication using `jose` (for Edge Runtime compatibility) and `bcryptjs` for secure password hashing.

---

3. Rubric Implementation & Features

A. Authentication, Security & Session Management

- **End-to-End Auth Flow:** The system features a fully functional signup and login flow. Sessions are managed using JSON Web Tokens (JWTs) stored in `httpOnly`, `secure`, and `sameSite: 'lax'` cookies.
- **Password Encryption:** Plain-text passwords are never stored or logged. All passwords (both user accounts and private room passwords) are hashed using `bcryptjs` with 12 salt rounds before database insertion.
- **Secure Comparison:** Login and private room entry rely strictly on secure hash-comparison functions (`bcrypt.compare`), never string equality.
- **Session Expiry:** JWTs are configured to expire after 7 days of inactivity, automatically requiring re-authentication.

B. Role-Based Access Control (Admin & User)

- **Distinct Roles:** The MongoDB `User` schema enforces `admin` and `user` roles.
- **Middleware Protection:** Next.js Edge Middleware (`proxy.ts`) intercepts all routing. If a standard user attempts to access `/admin` or any `api/admin/*` route, they are immediately blocked and redirected to a custom `403 Forbidden` error page.
- **Admin Dashboard:** Authorized admins have access to a dedicated dashboard where they can view the entire user list, toggle account activation status (suspend/activate), and change user roles in real-time.
- **Dynamic Navigation:** The frontend seamlessly adapts to the user's role. The "Admin" navigation link and footer links are conditionally rendered only if the decoded JWT payload contains `role: 'admin'`.

C. Real-Time Multiplayer Functionality & Game Logic

- **Canvas Synchronization:** The HTML5 `<canvas>` tracks mouse and touch events, emitting coordinates via Socket.IO. Late-joiners are instantly caught up via a base64 canvas snapshot (`toDataURL`) stored on the server, avoiding the need to replay thousands of individual strokes.
- **Turn Scheduling & Fairness:** The Socket server acts as the source of truth, managing timers, turn rotation, and word selection. It prevents race conditions by tracking exact guess times; the drawer cannot guess, and players who guess correctly are hidden from seeing other guesses to prevent cheating.
- **Room Management:** Players can join "Quick Play" (which auto-matches them to open public rooms) or create Custom Rooms. Custom rooms support configurable player limits, round counts, draw times, and optional password protection.
- **Live Friend System:** The lobby features a real-time friends panel where users can search for others, send requests, and accept invites. In the game room, users can open an Invite Modal to send push notifications to online friends to join their specific room.

D. UI / UX Design & Navigation

- **Aesthetic Polish:** Inkblot features a highly customized, cohesive design language utilizing a "glassmorphism" aesthetic, subtle paper-grain background textures, and playful typography (`Fredoka` for headings, `Caveat` for handwritten accents).
- **Responsive Layout:** The UI strictly adheres to mobile-first principles. CSS Grid layouts gracefully collapse into Flexbox columns on mobile devices (`max-width: 768px`). The drawing canvas is specifically engineered to maintain an exact `8/5` aspect ratio across all devices to ensure drawing coordinates translate perfectly between a mobile guesser and a desktop drawer.
- **Micro-interactions:** The application utilizes tasteful CSS animations, such as the pulsing "Quick Play" button, the animated SVG timer ring that turns pink under 10 seconds, and staggered letter reveals on the "Word of the Day" card.
- **Structure:** Sticky top navigation and a consistent footer with copyright and internal links are present across the application, ensuring no broken routes.

E. Form Validation

- **Client-Side:** HTML5 attributes and React state management enforce required fields, email formatting, and password input states before submission.
- **Server-Side:** API routes validate and sanitize payloads before interacting with MongoDB. Invalid requests (e.g., incorrect passwords, missing room IDs, taken usernames) return clear HTTP status codes and error messages, which the frontend displays inline to the user (e.g., pink error text below the Room ID input).

F. Git Version Control & Deployment

- **Version Control:** The project was iteratively developed using Git. It features over 15 meaningful commits following a semantic naming convention (e.g., `feat: implement realtime canvas`, `fix: mobile grid collapsing`, `docs: update setup instructions`).
- **Live Deployment:** The application is fully deployed and publicly accessible. The separation of the Vercel frontend and Railway backend ensures a seamless, zero-lag experience for end-users.

---

4. Key Technical Challenges Overcome

1. **Serverless WebSockets:** Initially, attempting to run Socket.IO on Vercel's serverless API routes resulted in constant disconnections. This was resolved by architecting a standalone Node.js server hosted on Railway, communicating securely with the Vercel frontend.
2. **CORS & Transport Protocols:** To ensure the Railway server accepted connections from the Vercel domain, strict CORS policies were configured. Furthermore, connection protocols were explicitly defined (`transports: ['websocket', 'polling']`) to ensure mobile

networks that block WebSockets could seamlessly fall back to HTTP polling without dropping the game.

3. **Canvas Coordinate Scaling:** Ensuring a line drawn on a large 1080p monitor appeared in the exact same relative position on a 400px wide smartphone screen required careful mathematical scaling (`canvas.width / rect.width`) alongside strict CSS aspect-ratio locking.
4. **State Race Conditions:** Solved a race condition where the drawer's word would occasionally not appear due to React state batching by forcing the server to send the target word directly inside the `round:start` payload for the drawer, eliminating the need for a secondary socket event.